

CSCI212 : Lecture #9

Kevin Bedoya

Wednesday 2/25/26

Review

The ArrayList

- The class ArrayList is a more flexible array

array	ArrayList
Declare:	Declare:
1 <code>String array[];</code>	1 <code>ArrayListString list;</code>
Create storage:	Create storage:
1 <code>array = new String[5];</code>	1 <code>list = new ArrayList<String>();</code>
Adding data:	Adding data:
1 <code>array[0] = "Hello"</code>	1 <code>list.add("Hello");</code>
Index based iteration:	Index based iteration:
1 <code>for(int i = 0; i < 10; i++) {</code> 2 <code>array[i] = "Hello";</code> 3 <code>}</code> 4 <i>// This will result in an error when i</i> 5 <i>= 5</i> 6 <i>// The array was initially fixed to a</i> 7 <i>length of 5 (strings)</i>	1 <code>for(int i = 0; i < 10; i++) {</code> 2 <code>list.add("Hello");</code> 3 <code>}</code>
Updating an element at an index:	Updating an element at an index:
1 <code>array[1] = "Goodbye";</code>	1 <code>list.add(1, "Goodbye");</code>

Downsides of ArrayLists:

- You can only use objects in an ArrayList:

```
1 ArrayList<int>
2 //This would be illegal in Java!
3 //Is there a work around?
4
```

Wrapper classes:

- *Wrapper class* **Integer** replaces **int**. Wherever you have an **int**, you can use **Integer**
- **autoboxing**: Java switches **int** to **Integer** as needed.

```
1 ArrayList<Integer> arrList = new ArrayList<Integer>();
2 // This is legal! Integer is an object analogous to the primitive int.
```

Subclasses & inheritance

So often you might code up a class, e.g. we might code:

```
1 public class Animal{
2     // Data about an animal
3     String name;
4     String species;
5     double weight;
6
7     // Actions
8     public String makeNoise(){
9         return "Growls";
10    }
11 }
```

If we then have to code up a **Dog**, we do not want to repeat all the code we have written for an **Animal** class however, we may specify some special properties unique to a **Dog** class. In Java, classes are able to *inherit* features from a *parent* class. *Inheritance* between classes, namely between a *parent* (or *super*) class and *child*(or *sub*) class, is possible with the **extends** keyword in Java .

For instance, we can write:

```
1 public class Dog extends Animal{
2     // unique dog features are included in here.
3 }
```

The *sub-class* inherits all features of the *super-class*, but has extra features we can code. A sub-class object is an object of a super-class. A sub-class produces an is a relationship between types.

We might want to recode (*override*) methods of the super-class.

```
1 public class Dog extends Animal{
2
3     public String makeNoise(){
4         return "Woof";
5     }
6     // Although any animal can make a noise,
7     // a dog make its own noise
8 }
```

End of lecture #9.